Computation for Analytics

DIANE WOODBRIDGE, PH.D

Contents

Python Data Types

Conditional Statements

Loop

Contents

Python Data Types

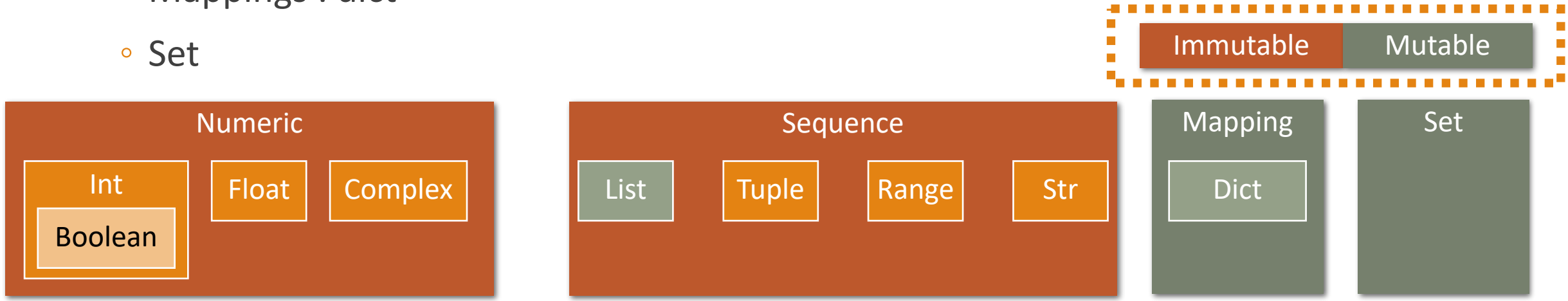
Conditional Statements

Loop

Python Data Types

Built-in Types (You can check by *type(var)*)

- Numerics : int, float, complex
- Sequences : list, tuple, range, str
- Mappings : dict
- Set

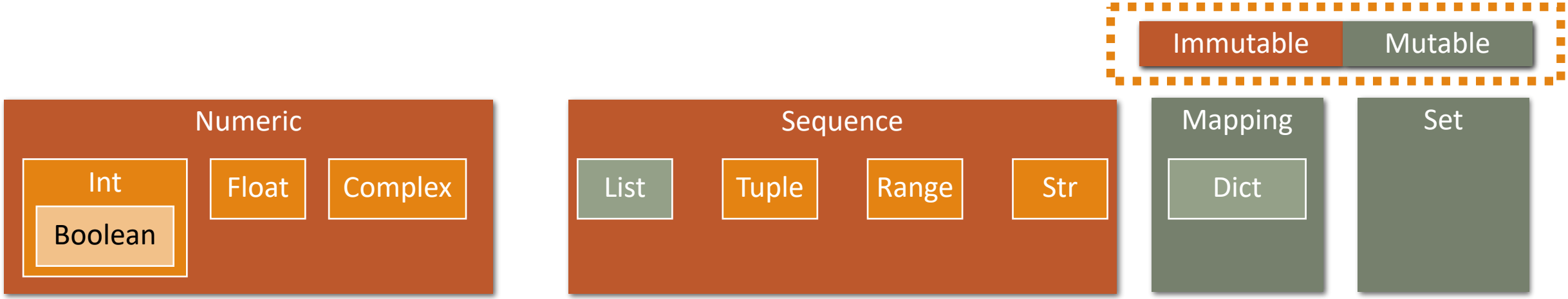


https://docs.python.org/3/library/stdtypes.html

Python Data Types

Built-in Types

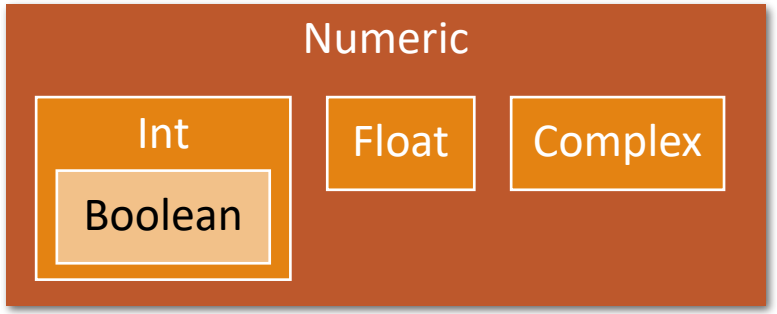
- Mutable : Types that you can change the value of a variable after its creation
- Immutable :Types that you cannot change the value of a variable after its creation



ex-mutable_vs_immutable.ipynb

https://docs.python.org/3/library/stdtypes.html

Python Data Types

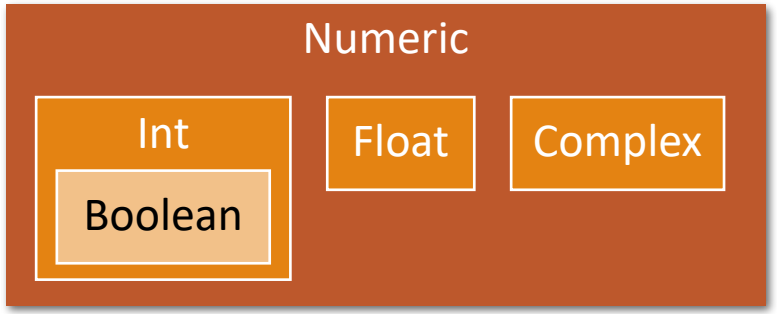


Numerics

- Types
  - Int (Integer)
    - Booleans are a subtype of integers
      - True : True, 1
      - False : False, 0, Some built-in objects including None, empty sequences and collections like [], {}, etc.
  - Float (Floating Point Number)
  - Complex (Complex Number) - Number with real and imaginary part (x + yi)

https://docs.python.org/3/library/stdtypes.html

Python Data Types



Numerics

- Built-in Operations

Operation	Results	Examples

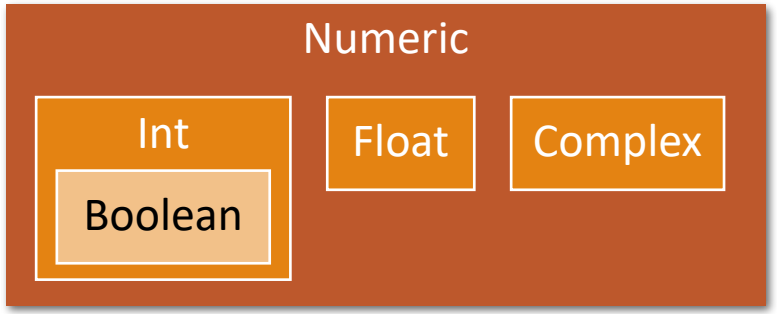
x[+, -, *, /]y	Basic Arithmetic Operations	`x = 6.52`
		`y = 5`
		`x+y # 11.52`

x//y	floored quotient of x and y	`x//y # 1.0`
x%y	Remainer of x/y	`x%y # 1.5199999999999996`
abs(x)	Absolute value of x	`abs(x) # 6.52`
int(x)	x converted to int	`int(x) # 6`
float(x)	x converted to float	`float(x) # 6.52`
complex(re, im=0)	A complex number with re as real and im as imaginary parts.	`complex(x, y) # (6.52+5j)`
pow(x,y), x**y	x to the power of y	`pow(x, y) #  11782.514385203196`
round(x,n=0)	x rounded to n digits	`round(x, 1) #6.5`

https://docs.python.org/3/library/stdtypes.html

ex_operations_examples.ipynb

Python Data Types



Numerics

- math Library Operations
  - Requires `import math`

Operation	Results	Examples

math.trunc(x)	Truncate x to integer	`x = 6.52`
		`y = 5`
		`base = 2`
		`math.trunc(x) # 6`

math.floor(x)	The greatest integer <= x	`math.floor(x) # 6`
math.ceil(x)	The least integer >= x	`math.ceil(x) # 7`
math.exp(x)	Return e raised to the power x	`math.exp(x) # 678.5783853394423`
math.log(x, [base])	Return the logarithm of x to the given base. Default for base is e	`math.log(y, base) # 2.321928094887362`

ex_operations_examples.ipynb

https://docs.python.org/3/library/stdtypes.html
https://docs.python.org/3/library/math.html

Example 1

If you are financing including mortgages/loans, you need to pay compound interests which including your principal and accumulated interest. To calculate the ending balance for compound interest, the equation is following.

$$Ending_Balance = Principal \times (1 + \frac{APR}{Yearly_Interest_Frequency})^{Years \times Yearly_Interest_Frequency}$$

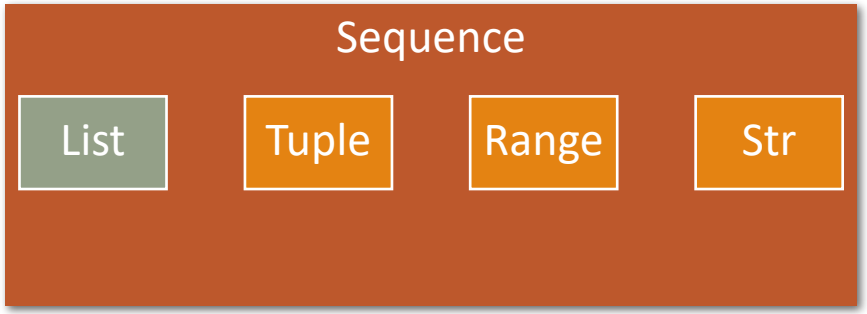
Complete ending_balance(principal, apr, years, interest_frequency) which returns the ending balance.

- You can assume that apr is given as %.

Ex. When principal = 20000, apr = 6, years = 3, interest_frequency = 12, ending_balance(principal, apr, years, interest_frequency) returns 23933.610496468293.

https://www.rocketmortgage.com/learn/compound-interest

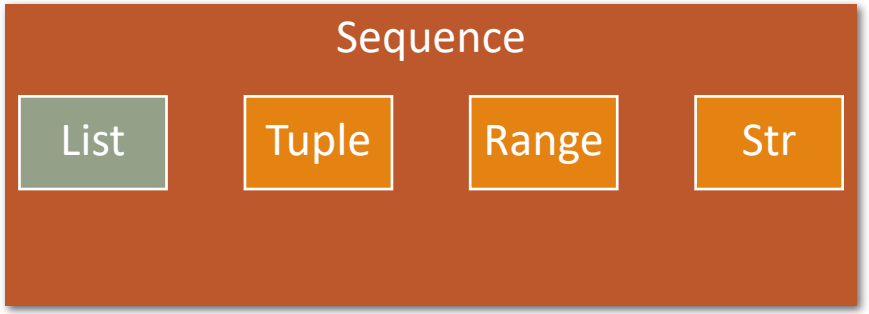
Python Data Types



Sequence

- Types
  - list
  - tuple
  - range
  - str - Text sequence type.
- Sequence supports **index** which indicates the location in a sequence.
  - Starts from 0. It can be a positive/negative number.

Python Data Types



Sequence

- Operations
  - When s and t are sequences, n,i,j, and k are integers, and x is an object of s.

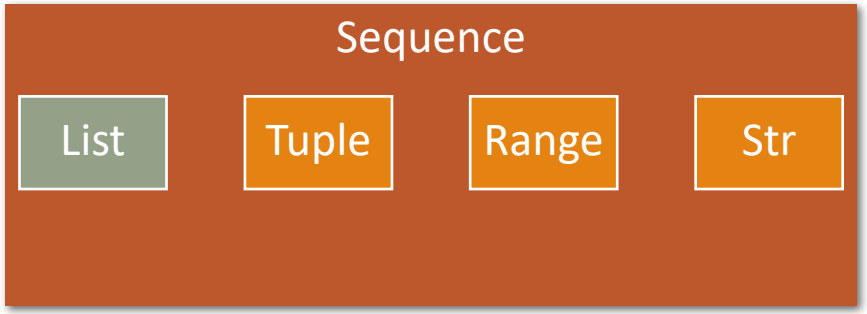
Operation	Results	Examples

x (not) in s	True (False) if an item of s is equal to x	`x = 2`
		`s, t = [1, 2, 3, 4], [5, 6]`
		`n,i,j,k = 2,1,3,2`
		`x in s # True`

s + t	Concatenation of s and t	`s+t # [1, 2, 3, 4, 5, 6]`
s * n (n * s)	Adding s to itself n times	`s*n # [1, 2, 3, 4, 1, 2, 3, 4]`
s[i]	(i+1)th item of s	`s[i] # 2`
s[i:j]	Slice of s from i to j	`s[i:j] # [2, 3]`
s[i:j:k]	Slice of s from i to j with step k	`s[i:j:k] # [2]`
len(s)	Length of s	`len(s) # 4`
min(s), max(s)	Smallest/largest item of s	`min(s) # 1` `max(s) # 4`
s.index(x)	Index of the first occurrence of x in s	`s.index(x) # 1`
s.count(x)	Total number of occurrences of x in s	`s.count(x) # 1`

ex_operations_examples.ipynb

Python Data Types

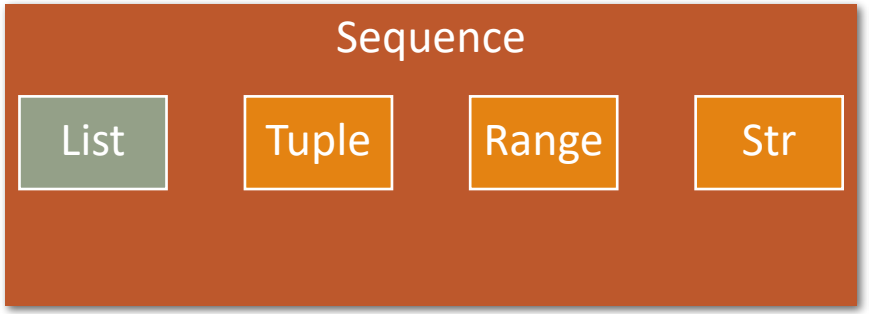


Sequence - List

- **Mutable** sequence which is a collection of items
- Constructed by
  - Using square brackets, separating items with commas: [], [a], [a, b, c]
  - Using a list comprehension: [x for x in iterable]
  - Using the type constructor: list() or list(iterable)
  - Also, you can create a list of list (ex. Matrix)

ex_list_comprehension.ipynb

https://docs.python.org/3/library/stdtypes.html#list



Python Data Types

Sequence - List

- **Mutable** sequence which is a collection of items
- Additional Operators

Operation	Results	Examples

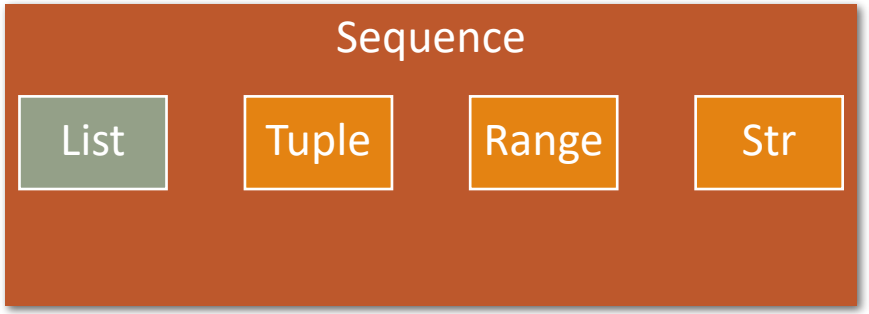
s.sort(key=None, reverse=False)	Sorted list based on ***key*** function which extract comparison key from each element (ex.key=str.lower) in the order of ***reverse*** (True/ False)	`s = [2, 1, 4, 3]`
		`i, x = 1, 4`
		`s.sort(reverse=True) # s becomes [4, 3, 2, 1]`

s.append(x)	Append x to the end of s	`s.append(x) # s becomes [2, 1, 4, 3, 4]`
s.insert(i, x)	Insert x into s at the index given by i	`s.insert(i, x) # s becomes [2, 4, 1, 4, 3]`
s.remove(x)	Remove the first item from s where its value is x	`s.remove(x) # s becomes [2, 1, 3]`
s.pop(i)	Retrieve the item at i and remove it from s	`s.pop(i) # s becomes [2, 4, 3]`
s.reverse()	Reverse the items of s in place	`s.reverse() # s becomes [3, 4, 1, 2]`

ex_operations_examples.ipynb

https://docs.python.org/3/library/stdtypes.html#list

Python Data Types



Sequence - List

- List Comprehension - Create a new list from (optionally filtered) elements in sequences
  - Syntax
    - [*expression* for *vars* in *elements* if *condition*]

```
seq = [0, 1, 2, 3, 4, 5]
```

```
even_sequence = [i for i in seq if i%2 ==0]
```

```
even_sequence
```

```
[0, 2, 4]
```

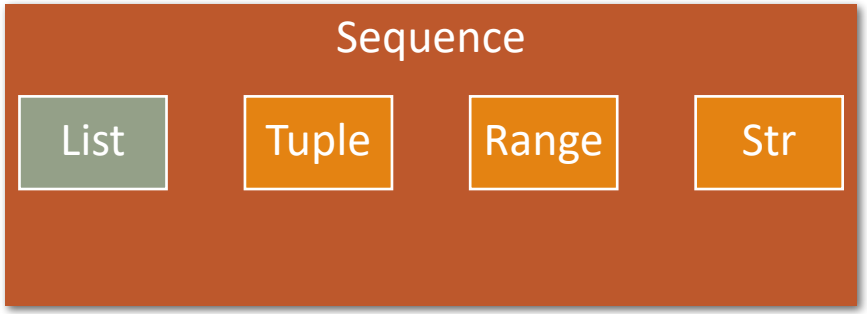
```
odd_sequence = [i for i in seq if i%2 !=0]
```

```
odd_sequence
```

```
[1, 3, 5]
```

https://docs.python.org/3/library/stdtypes.html#list

Python Data Types



Sequence - Tuple

- **Immutable** sequence which is a collection of items
- Constructed by
  - Using a pair of parentheses to denote the empty tuple: ()
  - Using a trailing comma for a singleton tuple: a, or (a,)
  - Separating items with commas: a, b, c or (a, b, c)
  - Using the tuple() built-in: tuple() or tuple(iterable)

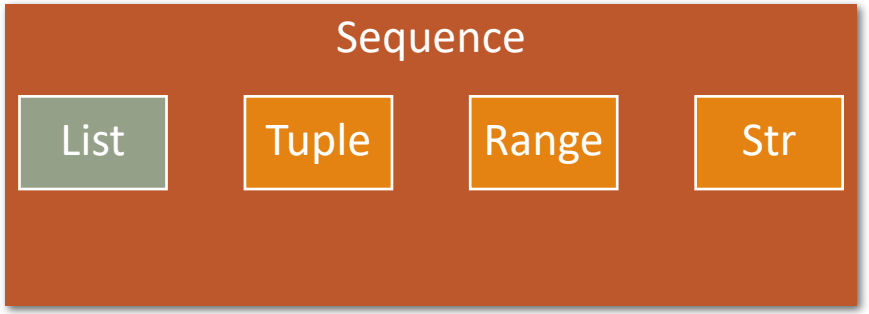
Example 2

Given a sorted list of integers, create a function called remove_elements() which returns a sorted array without any duplicates by removing them in place.

Ex. nums = [1, 1, 2], remove_elements(nums) should changes nums to [1,2]

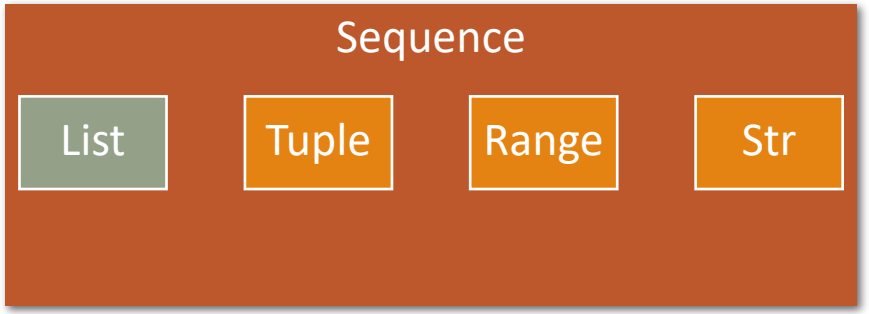
https://leetcode.com/problems/remove-duplicates-from-sorted-array/description/

Python Data Types



Sequence - Range

- **Immutable** sequence of numbers and is commonly used in for loops.
- Constructed by
  - range(start=0, stop[, step=1]) : sequence of numbers from ***start*** to ***stop***, incremented by step
- Advantage : Always take the same amount of memory, no matter the size of the range it represent compared to tuple and list.



Python Data Types

Sequence - String

- **Immutable** sequences of Unicode.
- Operations : Supports sequence operators and additional methods.

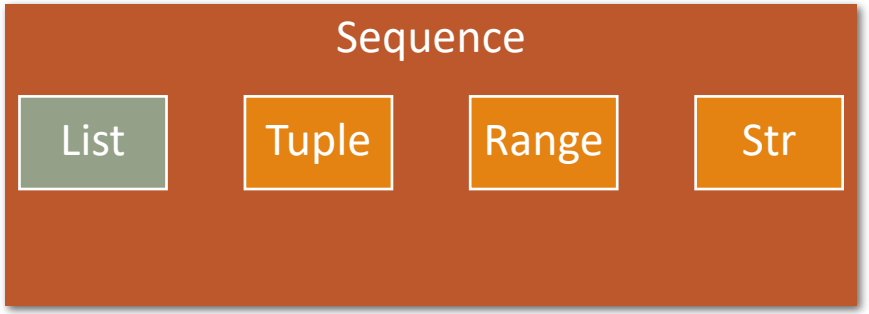
Operation	Results	Examples

str.endswith(suffix)
str.startswith(prefix)
	Return True if the string ends/starts with the specified suffix/prefix.	`str = " Hello, Everyone!"`
		`suffix, prefix, sub = "!", “Hello”, “one”`
		`str.endswith(suffix) # True`
		`str.startswith(prefix) # False`

str.find(sub)	Return the lowest index in the string where substring sub is found. If not found, return -1	`str.find(sub) # 13`
str.islower(), str.isupper()	Return True if all characters in str are lower/upper cases.	`str.islower() # False` `str.isupper() # False`
str.isalpha(), str.isnumeric()	Return True if all characters in the string are alphabetic/numeric.	`str.isalpha() # False` `str.isnumeric() # False`

ex_operations_examples.ipynb

Python Data Types



Sequence - String

- **Immutable** sequences of Unicode.
- Operations : Supports sequence operators and additional methods.

Operation	Results	Examples

str.lower(), str.upper()	Return all characters in str in lower/ upper cases	`str = " Hello, Everyone!"`
		`chars, sep =“ ", ","`
		`old, new = "Everyone", "Everybody"`
		`str.lower() # “ hello, everyone!”`
		`str.upper() # “ HELLO, EVERYONE!”`

str.strip([chars])	Return str with the leading and trailing characters removed. When chars is not given, it removes white spaces.	`str.strip(chars) # “Hello, Everyone!”`
str.split(sep=None)	Return a list of the words in the string, using sep as the delimiter string.	`str.split(sep) # [' Hello', ' Everyone!’]`
str.replace(old, new)	Return all occurrences of substring old replaced by new	`str.replace(old, new) # “ Hello, Everybody!”`

ex_operations_examples.ipynb

Example 3

For a string s and an integer k, reverse the first k characters in s for every 2k characters.

Ex. s = "abcdefg", k = 2, reverse_str(s,k) should return "bacdfeg"

https://leetcode.com/problems/reverse-string-ii

Example 4

For given two tuples, tuple1 and tuple2, create a function max_among_tuples() which returns the larger numbers on each index.

- If the length of tuple1 and tuple2 are different, the returned output should have the same length as the shorter one.
  - Ex. For tuple1 = (10, 4, 5) and tuple2 = (2, 5, 18), max_among_tuples(tuple1, tuple2) == (10, 5, 18)
  - Ex. For tuple1 = (1, 8, 13, 24) and tuple2 = (1, 10), max_among_tuples(tuple1, tuple2) == (1, 10)

https://www.geeksforgeeks.org/python-index-maximum-among-tuples/



Python Data Types

Mapping - Dict

- Mutable key-value pair structure
  - Key should be an immutable type, while values can be any type.
  - Ex. {“key1”:”value1”, “key2”:”value2”}, dict(key1=value1, key2=value2)
- Operations

Operation	Results	Examples

list(d)	list of all the keys in d	`d = {"A":"a", "B":"b", "C":"c"}`
		`k = "C"`
		`value = "c!"`
		`list(d) # ['A', 'B', 'C']`

d[k]	Return the item of d with key, k	`d[k] # c`
d[k] = value	Set d[k] to value.	`d[k] = value # d[k] becomes c!`
iter(d)	Return an iterator over the keys of d	`iter(d) # <dict_keyiterator object>`
d.keys(), d.values()	Return d’s keys / values.	`d.keys() # ['A', 'B', 'C']` `d.values() # ['a', 'b', 'c']`
d.items()	Return d’s (key, value) pairs.	`d.items() # [('A', 'a'), ('B', 'b'), ('C', 'c')]`

ex_operations_examples.ipynb



Python Data Types

Set

- Create by set(), {elem1, elem2, …}
- Mutable unordered collection of distinct objects
  - Useful for : membership testing, removing duplicates from a sequence

Operation	Results	Examples

s.add(item)	Add element item to the set, s.	`s = {1, 2, 2, 3}`
		`item = 4`
		`x = 4`
		`other = {2, 3}`
		`s.add(item) # s becomes {1, 2, 3, 4}`

s.remove(item)	Remove element item from the set, s.	`s.remove(item) # s becomes {1, 2, 3}`
x [not] in s	Test x for [non-]membership in s.	`x in s # False`
s.isdisjoint(other)	Return True if the s has no elements in common with other.	`s.isdisjoint(other) # False`

ex_operations_examples.ipynb

Set

Python Data Types

Set

- Create by set(), {elem1, elem2, …}
- Mutable unordered collection of distinct objects
  - Useful for : membership testing, removing duplicates from a sequence

Operation	Results	Examples
s.issubset(other), s<= other	Return True if every element in the s is in other.	`s.issubset(other) # False`
s.issuperset(other), s>= other	Return True if every element in other is in the s.	`s.issuperset(other) # True`
s.union(other)	Return a new set with elements from s and others.	`s.union(other) # {1, 2, 3}`
s.intersection(others)	Return a new set with elements common to s and others.	`s.intersection(other) # {2, 3}`
s.difference(others)	Return a new set with elements in the set that are not in the others.	`s.difference(other) # {1}`

ex_operations_examples.ipynb

Example 5

For a list of strings in a format of "category-courseID", complete class_dictionary() that creates a dictionary of "category":"set of unique courseIDs".

- Ex. classes = ["computation-MSDS597","statistics-MSDS504","computation-MSDS501", "computation-MSDS501", "statistics-MSDS504"], classDictionary(classes) returns {'computation': {'MSDS501', 'MSDS597'}, 'statistics': {‘MSDS504'}}
- From the created dictionary, return all the "computation" courses in a list.

Contents

Python Data Types

Conditional Statements

Loop

Conditional Statements

If statement

- Checks whether the condition in “if” is true to execute the statements. Otherwise checks ***elif*** (can have more than 0) and then ***else*** (optional) statements.
- Syntax

```
if condition :
  statements
elif condition:
  statements
else:
  statements
```

Contents

Python Data Types

Conditional Statements

Loop

Loops

while

- Syntax : while(condition):
- Executes as long as the condition remains true.

for

- Syntax : for it in sequence:
- Iterates over the items in the sequence.

Some useful statements for loops

- break : Breaks out of the innermost enclosing loop
- else : Executed not when the loop is terminated by break
- continue : Continues with the next iteration of the loop

Loops

Iterator

- Implementation for iteration over containers
  - User can define a class to support iterations.
  - Sequence types always support iterations
  - Iterators should be able to return the iterator itself or the next item from the iterator until there is no more.

Generator

- Conveniently create an iterator object with the iterator itself and the next item.
- Need to use **`yield`** as a key word

See Example 6.

Example 6

Class Power requires 2 seconds for initializing an object.

- Let's create a function that takes nums (integers between 0 and 9) and val, creates a Power object and apply the apply_power method using 1) generator and 2) iterator.
- Which one was faster?

```
class Power:
    def __init__(self, p):
        """
        initialization takes 2 second
        """
        self.p = p
        time.sleep(2)

    def apply_power(self, x):
        return x**self.p
```

Useful for Conditional/Loop

Pass

- The `pass` statement is for doing nothing.
  - It is useful when a statement is required syntactically, but your program does not require an action.
  - Useful for conditional and loop statements.

Example 7

For nums (an array of integers) and target, return a list of two indices where the sum of the corresponding numbers is equal to target.

Ex. For nums = [2,7,11,15], target = 9, two_sum(nums, target) returns [0,1].

https://leetcode.com/problems/two-sum/

Contents

Python Data Types

Conditional Statements

Loop

Extra Questions

Geeks for geeks - https://www.geeksforgeeks.org/python-exercises-practice-questions-and-solutions/

Leetcode : https://leetcode.com/problemset

References

Python : https://docs.python.org/3/